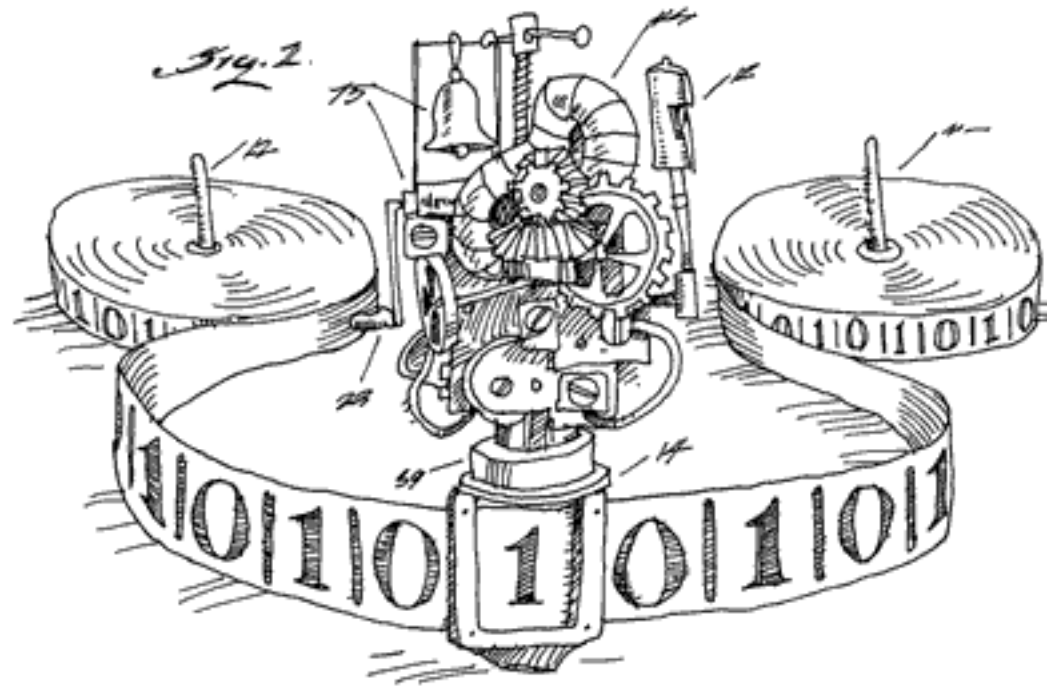


CSC4010: Introduction to Theoretical Computer Science



Lecture 8

Adapted from Sipser's MIT OCW Course on TOC

Last Time

1) Turing Machines, via CT Thesis,
this captures all computational models

→ was a useful problem?

- Is there an explicit computational problem which cannot be solved in any computational ~~problem~~?

model.

→ $\{ \langle M, w \rangle \mid M \text{ accepts } w \}$

- We showed that L_{self} , and A_{TM} are undecidable via diagonalization.

→ $\{ \langle M \rangle \mid M \text{ rejects } \langle M \rangle \}$

- A language L is decidable if and only if L and $\bar{L}$ are both recognizable. We used this to conclude that $\bar{A}_{TM}$ is unrecognizable.

→ not true, then, A_{TM} would be decidable.

- We showed HALT_{TM} is undecidable by reducing A_{TM} to HALT_{TM} .

- Proof Strategy:** If there is a decider H for HALT_{TM} , then we can construct a decider M' for A_{TM} . But A_{TM} is undecidable.

→ M' can't exist

→ H cannot exist $\Rightarrow \text{HALT}_{TM}$ is undecidable.

Reducibility Method: Undecidability of HALT_{TM}

- Let $\text{HALT}_{TM} = \{ \langle M, w \rangle \mid M \text{ halts on } w \}$.

Reducibility Method: Undecidability of HALT_{TM}

- Let $\text{HALT}_{TM} = \{ \langle M, w \rangle \mid M \text{ halts on } w \}$.

Theorem: HALT_{TM} is undecidable.

Reducibility Method: Undecidability of HALT_{TM}

- Let $\text{HALT}_{TM} = \{ \langle M, w \rangle \mid M \text{ halts on } w \}$.

Theorem: HALT_{TM} is undecidable.

$\hookrightarrow H$ halts on all inputs.
 $\hookrightarrow H(\langle M, w \rangle) = \text{accept} \Leftrightarrow \langle M, w \rangle \in \text{HALT}_{TM}$

- Proof:** Proof by contradiction. Assume there exists H deciding HALT_{TM} .

Reducibility Method: Undecidability of HALT_{TM}

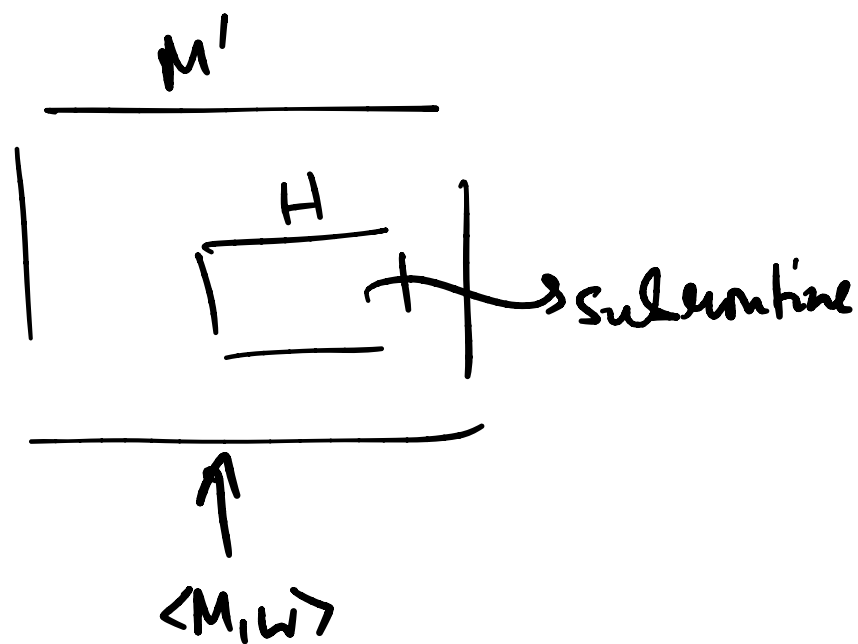
- Let $\text{HALT}_{TM} = \{ \langle M, w \rangle \mid M \text{ halts on } w \}$.

Theorem: HALT_{TM} is undecidable.

$\hookrightarrow H$ halts on all inputs.
 $\hookrightarrow H(\langle M, w \rangle) = \text{accept} \Leftrightarrow \langle M, w \rangle \in \text{HALT}_{TM}$

- Proof:** Proof by contradiction. Assume there exists H deciding HALT_{TM} .

Idea: Use H to construct a Turing Machine M' deciding A_{TM} .



i) Suppose H is run on $\langle M, w \rangle$.

- a) Accepts $\langle M, w \rangle$
- b) rejects $\langle M, w \rangle$

$\hookrightarrow \langle M, w \rangle$ is my input to A_{TM} .

$\Rightarrow M$ does not halt on w .
 $\Rightarrow \langle M, w \rangle \notin A_{TM}$

$\hookrightarrow H$ accepts $\langle M, w \rangle$
 $\hookrightarrow$ we know M halts on w .

We run M on w .
 a) M accepts w
 b) M rejects w .

Reducibility Method: Undecidability of HALT_{TM}

- Let $\text{HALT}_{TM} = \{ \langle M, w \rangle \mid M \text{ halts on } w \}$.

Theorem: HALT_{TM} is undecidable.

- Proof:** Proof by contradiction. Assume there exists H deciding HALT_{TM} .

Idea: Use H to construct a Turing Machine M' deciding A_{TM} .

$M' =$ "On input $\langle M, w \rangle$

1. Simulate H on input $\langle M, w \rangle$.
2. Reject if H rejects. (as $\langle M, w \rangle \notin A_{TM}$)
3. If H accepts, run M on w until it halts. $\Rightarrow$ we know M halts on w .
 - (a) If M accepts, then accept.
 - (b) If M rejects, then reject."

Reducibility Method: Undecidability of HALT_{TM}

- Let $\text{HALT}_{TM} = \{ \langle M, w \rangle \mid M \text{ halts on } w \}$.

c) M loops on w .
(i) Then H rejects $\langle M, w \rangle$.

Theorem: HALT_{TM} is undecidable.

- Proof:** Proof by contradiction. Assume there exists H deciding HALT_{TM} .

Idea: Use H to construct a Turing Machine M' deciding A_{TM} .

M' = "On input $\langle M, w \rangle$

1. Simulate H on input $\langle M, w \rangle$.
2. Reject if H rejects.
3. If H accepts, run M on w until it halts.
 - (a) If M accepts, then accept.
 - (b) If M rejects, then reject."

Claim: If H exists, then M' decides A_{TM} .

a) $\langle M, w \rangle \in A_{TM}$
in step 1, H accepts $\langle M, w \rangle$
in step 3a, M accepts w ,
and we also accept

b) M rejects w :
1a) H accepts $\langle M, w \rangle$
3b) M rejects w .

Reducibility - Concept

If we have an algorithm for $HALT_{TM}$
 $\Rightarrow$ we have an algorithm for A_{TM} .

- If we have two languages (or problems) A and B , then A being reducible to B implies we can use B to solve A .
 A_{TM} was reducible to $HALT_{TM}$
- **Example:** Measuring the area of a rectangle is reducible to measuring the lengths of its sides.
(green arrow from "lengths" to "area")
- If A is reducible to B , then solving B gives a solution to A .
 - Then, B is easy implies A is easy.
 - Then, A is hard implies B is hard.
 $\underbrace{\text{undecidable}}_{\text{unrecognizable}} \Rightarrow \underbrace{B}_{\text{undecidable}} \Rightarrow \text{unrecognizable.}$

Reducibility - Concept

- If we have two languages (or problems) A and B , then A being reducible to B implies we can use B to solve A .
- **Example:** Measuring the area of a rectangle is reducible to measuring the lengths of its sides.
- If A is reducible to B , then solving B gives a solution to A .
 - Then, B is easy implies A is easy.
 - Then, A is hard implies B is hard.

This is the form we will use.

Another Example: EMPTY_{TM}

Another Example: EMPTY_{TM}

- Let $\text{EMPTY}_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$. $\rightarrow M$ does not accept any string.

Another Example: EMPTY_{TM}

$\rightarrow M$ does not accept anything.

- Let $\text{EMPTY}_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$.

Theorem: EMPTY_{TM} is undecidable.

"Reduce A_{TM} to EMPTY_{TM} "
"which means, that we will show
that if $\neg$ an algorithm for EMPTY_{TM}
then $\neg$ an algorithm for A_{TM} ."

Another Example: EMPTY_{TM}

- Let $\text{EMPTY}_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$.
 - Theorem:** EMPTY_{TM} is undecidable.
 - Proof:** Proof by contradiction. Assume there exists R deciding EMPTY_{TM} .
 - $\uparrow$ "reduce A_{TM} to EMPTY_{TM} "
which means, that we will show
that if $\exists$ an algorithm for EMPTY_{TM}
then $\exists$ an algorithm for A_{TM} .
 $\langle M \rangle$ is i/p and it accepts iff
 $L(M) \neq \emptyset$.

Another Example: EMPTY_{TM}

- Let $\text{EMPTY}_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$.

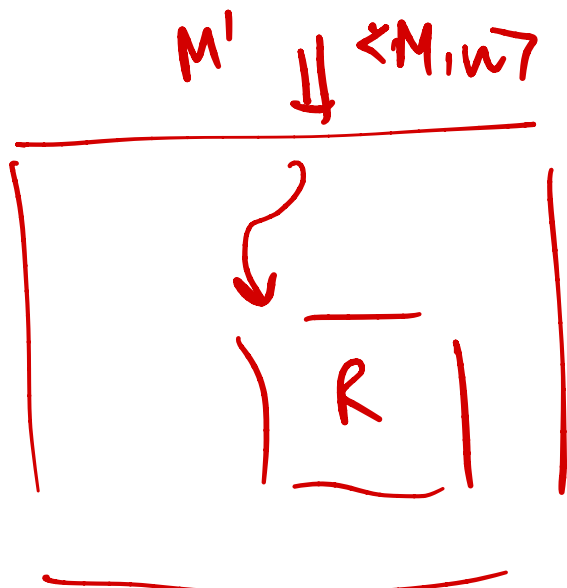
Theorem: EMPTY_{TM} is undecidable.

$\langle M \rangle$ is i/p and it accepts iff
 $\Downarrow L(M) \neq \emptyset$.

- Proof:** Proof by contradiction. Assume there exists R deciding EMPTY_{TM} .

Idea: Use R to construct a Turing Machine M' deciding A_{TM} .

$\hookrightarrow \langle M, w \rangle$ as input.



Attempt 1:

$\langle M \rangle$ through R .

a) R accepts $\langle M \rangle$: $L(M) \neq \emptyset$
 $\langle M, w \rangle \notin A_{TM}$.

b) R rejects $\langle M \rangle$: $L(M) \neq \emptyset$
 $w \in L(M)$ or $w \notin L(M)$?

Another Example: EMPTY_{TM}

- Let $\text{EMPTY}_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$.

Theorem: EMPTY_{TM} is undecidable.

- Proof:** Proof by contradiction. Assume there exists R deciding EMPTY_{TM} .

Idea: Use R to construct a Turing Machine M' deciding A_{TM} .

Attempt 1: Given input $\langle M, w \rangle$ for M' , we pass $\langle M \rangle$ through R .

Another Example: EMPTY_{TM}

- Let $\text{EMPTY}_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$.

Theorem: EMPTY_{TM} is undecidable.

- Proof:** Proof by contradiction. Assume there exists R deciding EMPTY_{TM} .

Idea: Use R to construct a Turing Machine M' deciding A_{TM} .

Another Example: EMPTY_{TM}

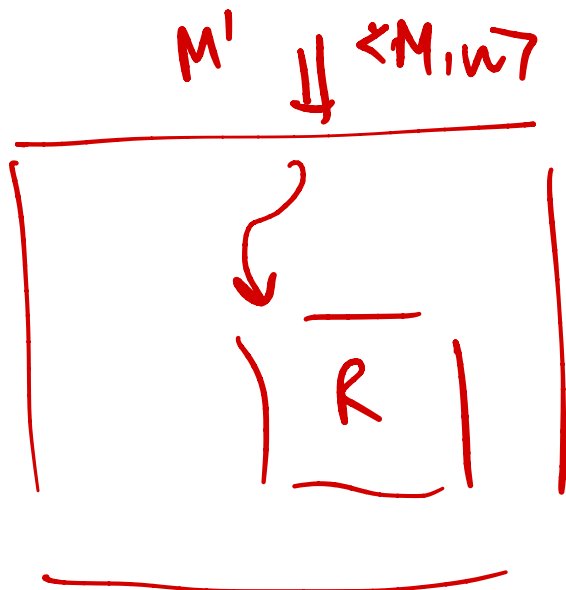
- Let $\text{EMPTY}_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$.

Theorem: EMPTY_{TM} is undecidable.

- Proof:** Proof by contradiction. Assume there exists R deciding EMPTY_{TM} .

Idea: Use R to construct a Turing Machine M' deciding A_{TM} .

Attempt 2: Given input $\langle M, w \rangle$ for M' , transform $\underline{M}$ to $\underline{M}_w$ defined as follows:



Attempt 1:

$\langle M \rangle$ through R .

a) R accepts $\langle M \rangle$: $L(M) \neq \emptyset$
 $\langle M, w \rangle \notin A_{TM}$.

b) R rejects $\langle M \rangle$: $L(M) \neq \emptyset$
 $w \in L(M)$ or $w \notin L(M)$?

$\hookrightarrow$ as i/p anything x .
 a) If $x \neq w$ then it rejects.

Another Example: EMPTY_{TM}

- Let $\text{EMPTY}_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$.

Theorem: EMPTY_{TM} is undecidable.

- Proof:** Proof by contradiction. Assume there exists R deciding EMPTY_{TM} .

Idea: Use R to construct a Turing Machine M' deciding A_{TM} .

Attempt 2: Given input $\langle M, w \rangle$ for M' , transform M to M_w defined as follows:

$M_w =$ "On input $x \in \{0,1\}^*$

1. If $x \neq w$, then reject.
2. Simulate M on w .

$\langle M \rangle, \langle w \rangle$
 $\Downarrow$
 $\langle M_w \rangle$

Another Example: EMPTY_{TM}

- Let $\text{EMPTY}_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$.

Theorem: EMPTY_{TM} is undecidable.

- Proof:** Proof by contradiction. Assume there exists R deciding EMPTY_{TM} .

Idea: Use R to construct a Turing Machine M' deciding A_{TM} .

Attempt 2: Given input $\langle M, w \rangle$ for M' , transform M to M_w defined as follows:

$M_w =$ "On input $x \in \{0,1\}^*$

1. If $x \neq w$, then reject.
2. Simulate M on w .

$$L(M_w) = \begin{cases} \{w\} & \text{if } M \text{ accepts } w \\ \emptyset & \text{if } M \text{ does not accept } w \end{cases}$$

Another Example: EMPTY_{TM}

- Let $\text{EMPTY}_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$.

Theorem: EMPTY_{TM} is undecidable.

$L(M_w) = \emptyset$
 $\Leftrightarrow M$ does not accept w
 $\Leftrightarrow \langle M, w \rangle \notin A_{TM}$

- Proof:** Proof by contradiction. Assume there exists R deciding EMPTY_{TM} .

Idea: Use R to construct a Turing Machine M' deciding A_{TM} .

Attempt 2: Given input $\langle M, w \rangle$ for M' , transform M to M_w defined as follows:

$M' =$ "On input $\langle M, w \rangle$

1. Transform $\langle M, w \rangle$ into M_w .

2. Run R on $\langle M_w \rangle$. $\rightarrow L(M_w) \neq \emptyset$

3. Accept if R rejects. $\Leftrightarrow \{w\} = L(M_w)$

4. Reject if R accepts. $\Leftrightarrow M$ accepts w
 $\Leftrightarrow \langle M, w \rangle \in A_{TM}$

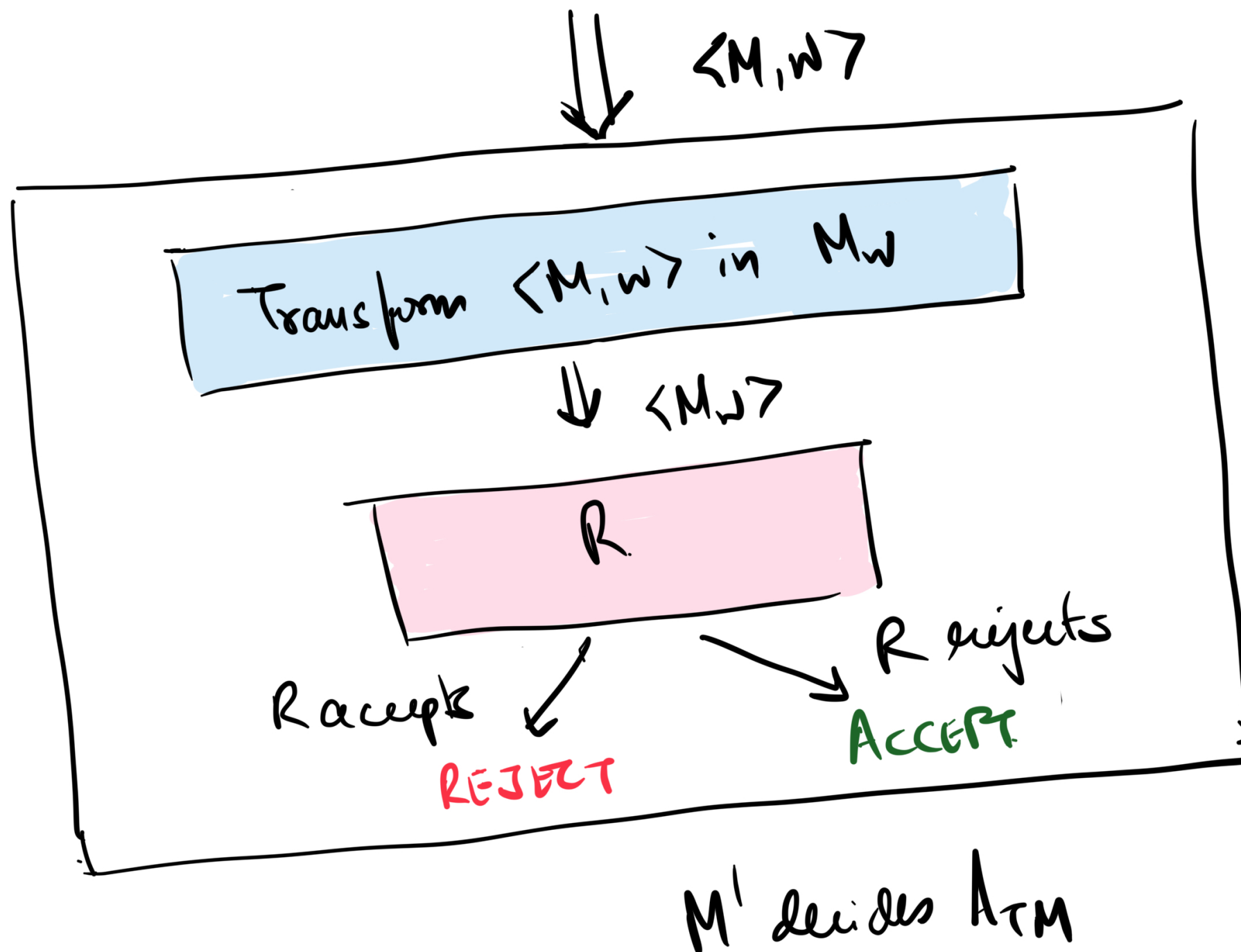
$M_w =$ "On input $x \in \{0,1\}^*$

1. If $x \neq w$, then reject.

2. Simulate M on w .

$L(M_w) = \begin{cases} \{w\} & \text{if } M \text{ accepts } w \\ \emptyset & \text{if } M \text{ does not accept } w \end{cases}$

Another Example: EMPTY_{TM}



Mapping Reducibility

Mapping Reducibility

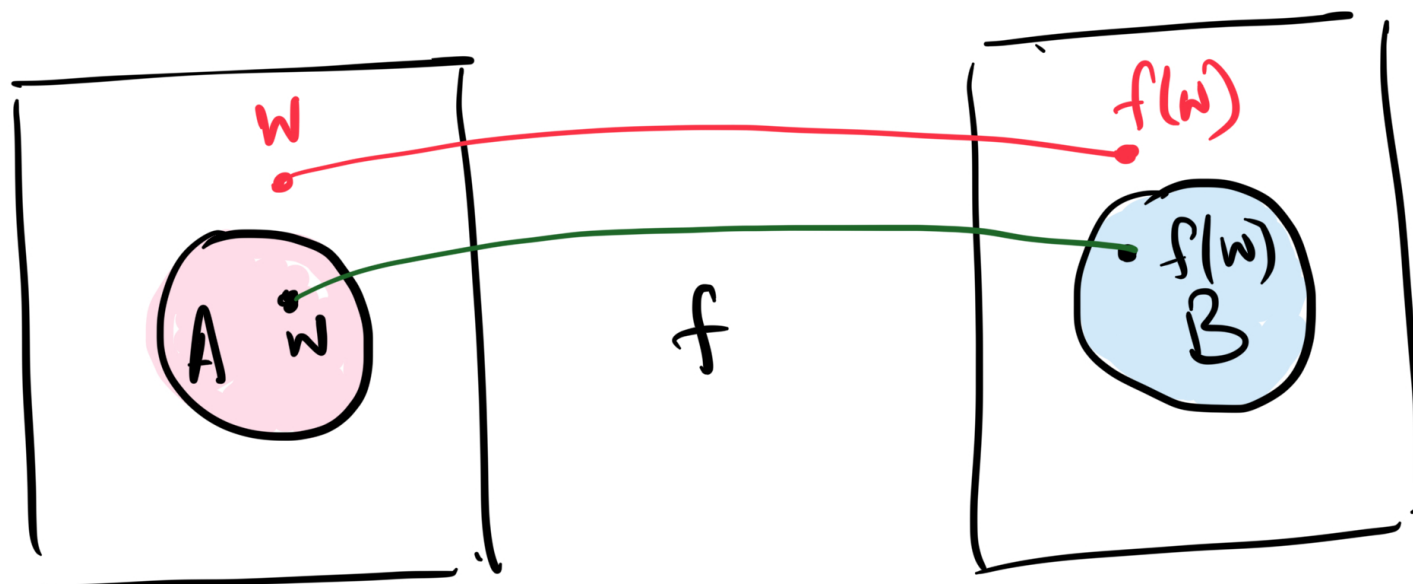
- **Definition:** A function $f : \Sigma^* \rightarrow \Sigma^*$ is said to be **computable** if there exists a Turing Machine F , which on input $w \in \Sigma^*$ terminates with just $f(w)$ written on its tape, for any w .

Mapping Reducibility

- **Definition:** A function $f : \Sigma^* \rightarrow \Sigma^*$ is said to be **computable** if there exists a Turing Machine F , which on input $w \in \Sigma^*$ terminates with just $f(w)$ written on its tape, for any w .
- **Definition:** A language A is **mapping reducible** to a language B (denoted $A \leq_m B$) if and only if there exists a computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that for every $w \in \Sigma^*$, $w \in A \iff f(w) \in B$.

Mapping Reducibility

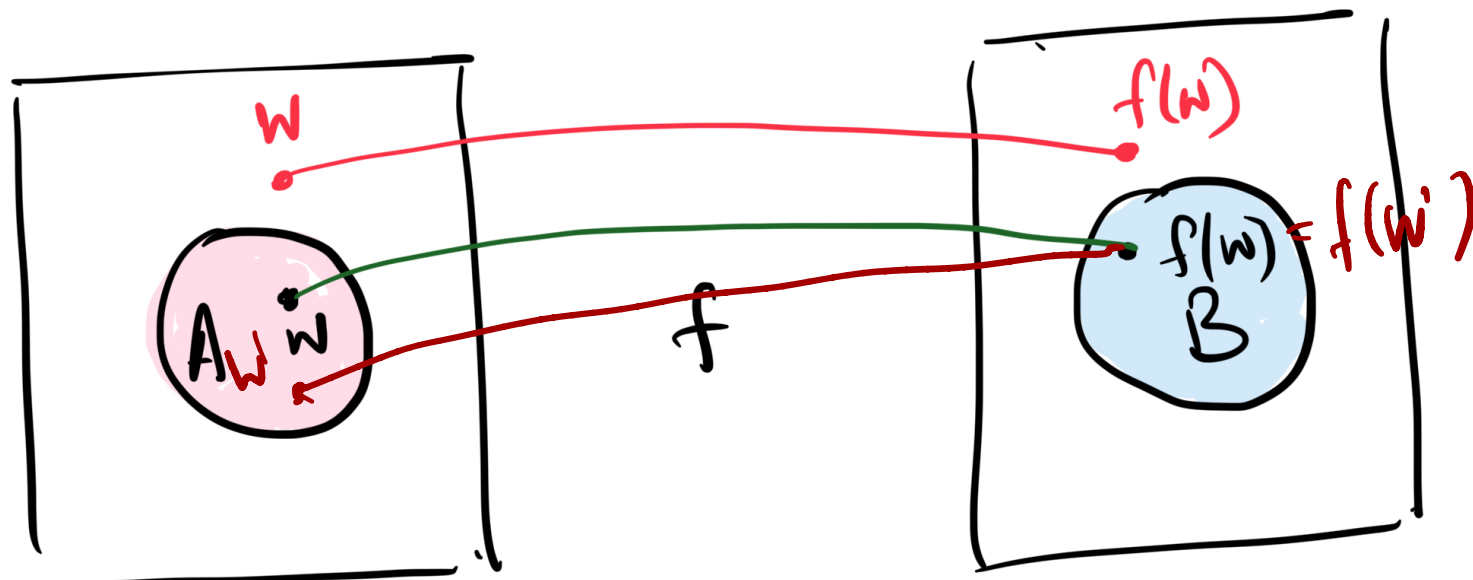
- **Definition:** A function $f : \Sigma^* \rightarrow \Sigma^*$ is said to be **computable** if there exists a Turing Machine F , which on input $w \in \Sigma^*$ terminates with just $f(w)$ written on its tape, for any w .
- **Definition:** A language A is **mapping reducible** to a language B (denoted $A \leq_m B$) if and only if there exists a computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that for every $w \in \Sigma^*$, $w \in A \iff f(w) \in B$.



∃ a Turing machine which transforms an instance of A into an instance of B .

Mapping Reducibility

- Definition:** A function $f: \Sigma^* \rightarrow \Sigma^*$ is said to be **computable** if there exists a Turing Machine F , which on input $w \in \Sigma^*$ terminates with just $f(w)$ written on its tape, for any w .
- Definition:** A language A is **mapping reducible** to a language B (denoted $A \leq_m B$) if and only if there exists a computable function $f: \Sigma^* \rightarrow \Sigma^*$ such that for every $w \in \Sigma^*$, $w \in A \iff f(w) \in B$.
 → Suppose we know how to solve B .
 → we want to solve A .



f is called a mapping reduction from A to B .

need a TM to decide A .
 $x \in \{0,1\}^* \rightarrow f(x) \Rightarrow M_B$
 if $M_B(f(x)) = 1 \iff x \in A$

$A \leq_m B$ { we know how to solve B .
 (1) solve A by translating its instances to B .

$$A_{TM} \leq_m \overline{\text{EMPTY}_{TM}} \Rightarrow \{ \langle M, w \rangle \mid M \text{ accepts } w \} \Rightarrow \{ \langle M, \gamma \rangle \mid L(M) \neq \emptyset \}.$$

$$\textcircled{1} \xrightarrow{\text{mapping reducible to}} A_{TM} \leq_m \overline{\text{EMPTY}}_{TM}$$

$$f : \Sigma^* \rightarrow \Sigma^* \text{ (computable)}$$

$$x \in A_{TM} \Leftrightarrow f(x) \in \overline{\text{EMPTY}}_{TM}.$$

Solution: Given input $\langle M, w \rangle$ for A_{TM} , $f(\langle M, w \rangle) = \langle M_w \rangle$ defined as follows:

$$A_{TM} \leq_m \overline{\text{EMPTY}}_{TM}$$

→ Computable as $\exists TM$ that on i/p $\langle M, w \rangle$ outputs $\langle M, w \rangle$.

$f: \Sigma^* \rightarrow \Sigma^*$

Solution: Given input $\langle M, w \rangle$ for A_{TM} , $f(\langle M, w \rangle) = \langle M_w \rangle$ defined as follows:

$$\langle M, w \rangle \in A_{TM} \Leftrightarrow \underbrace{f(\langle M, w \rangle) \in \overline{\text{EMPTY}}_{TM}}.$$

$M_w =$ "On input $x \in \{0,1\}^*$

1. If $x \neq w$, then reject.
2. Simulate M on w .

$$A_{TM} \leq_m \overline{\text{EMPTY}}_{TM}$$

Solution: Given input $\langle M, w \rangle$ for A_{TM} , $f(\langle M, w \rangle) = \langle M_w \rangle$ defined as follows:

$M_w =$ "On input $x \in \{0,1\}^*$

1. If $x \neq w$, then reject.
2. Simulate M on w .

$$L(M_w) = \begin{cases} \{w\} & \text{if } M \text{ accepts } w \\ \emptyset & \text{if } M \text{ does not accept } w \end{cases}$$

$$\langle M, w \rangle \in A_{TM} \Leftrightarrow f(\langle M, w \rangle) \in \overline{\text{EMPTY}}_{TM}.$$

$$\begin{aligned} \langle M, w \rangle \in A_{TM} &\Leftrightarrow L(M_w) = \{w\} \\ &\Leftrightarrow L(M_w) \neq \emptyset \\ &\Leftrightarrow \langle M, w \rangle \in \overline{\text{EMPTY}}_{TM} \\ &\Leftrightarrow f(\langle M, w \rangle) \in \overline{\text{EMPTY}}_{TM}. \end{aligned}$$

$$A_{TM} \leq_m \overline{\text{EMPTY}}_{TM}$$

Solution: Given input $\langle M, w \rangle$ for A_{TM} , $f(\langle M, w \rangle) = \langle M_w \rangle$ defined as follows:

$M_w =$ "On input $x \in \{0,1\}^*$

1. If $x \neq w$, then reject.
2. Simulate M on w .

Claim: $f(\langle M, w \rangle) \in \overline{\text{EMPTY}}_{TM} \iff \langle M, w \rangle \in A_{TM}.$

$$L(M_w) = \begin{cases} \{w\} & \text{if } M \text{ accepts } w \\ \emptyset & \text{if } M \text{ does not accept } w \end{cases}$$

Mapping Reducibility: Properties

Theorem: If $A \leq_m B$ and B is decidable, then A is decidable.

Mapping Reducibility: Properties

Theorem: If $A \leq_m B$ and B is decidable, then A is decidable.

“If B is at least as hard as A , and B is decidable then so is A .”

Mapping Reducibility: Properties

Theorem: If $A \leq_m B$ and B is decidable, then A is decidable.

“If B is at least as hard as A , and B is decidable then so is A .”

- **Proof:** Let M_B be a decider for B . We build a decider for M_A for A as follows:

Mapping Reducibility: Properties

Theorem: If $A \leq_m B$ and B is decidable, then A is decidable.

“If B is at least as hard as A , and B is decidable then so is A .”

- **Proof:** Let M_B be a decider for B . We build a decider for M_A for A as follows:

$M_A =$ “On input $w \in \{0,1\}^*$

1. Compute $f(w)$.

2. Run M_B on $f(w)$ and output its answer.

f is computable, $\exists$ an algorithm that computes $f(w)$.

$\Rightarrow A \leq_m B$ so $\exists$ subarray which is a mapping function from A to B .

Mapping Reducibility: Properties

Theorem: If $A \leq_m B$ and B is decidable, then A is decidable.

“If B is at least as hard as A , and B is decidable then so is A .”

- **Proof:** Let M_B be a decider for B . We build a decider for M_A for A as follows:

M_A = “On input $w \in \{0,1\}^*$

1. Compute $f(w)$.
2. Run M_B on $f(w)$ and output its answer.

M_A accepts $w \iff M_B$ accepts $f(w)$ (By our algorithm)
 $\iff f(w) \in B$ (M_B is a decider for B)
 $\iff w \in A$. (mapping reduction)

Mapping Reducibility: Properties

Theorem: If $A \leq_m B$ and B is decidable, then A is decidable.

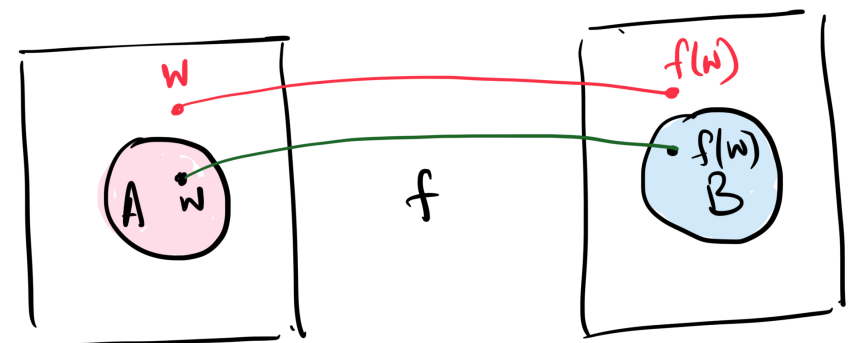
“If B is at least as hard as A , and B is decidable then so is A .”

- **Proof:** Let M_B be a decider for B . We build a decider for M_A for A as follows:

M_A = “On input $w \in \{0,1\}^*$

1. Compute $f(w)$.
2. Run M_B on $f(w)$ and output its answer.

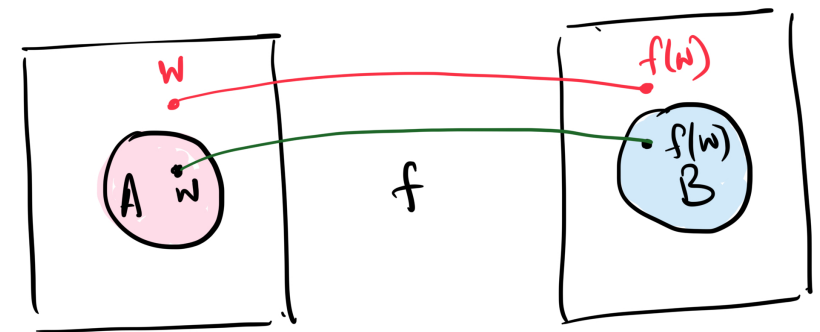
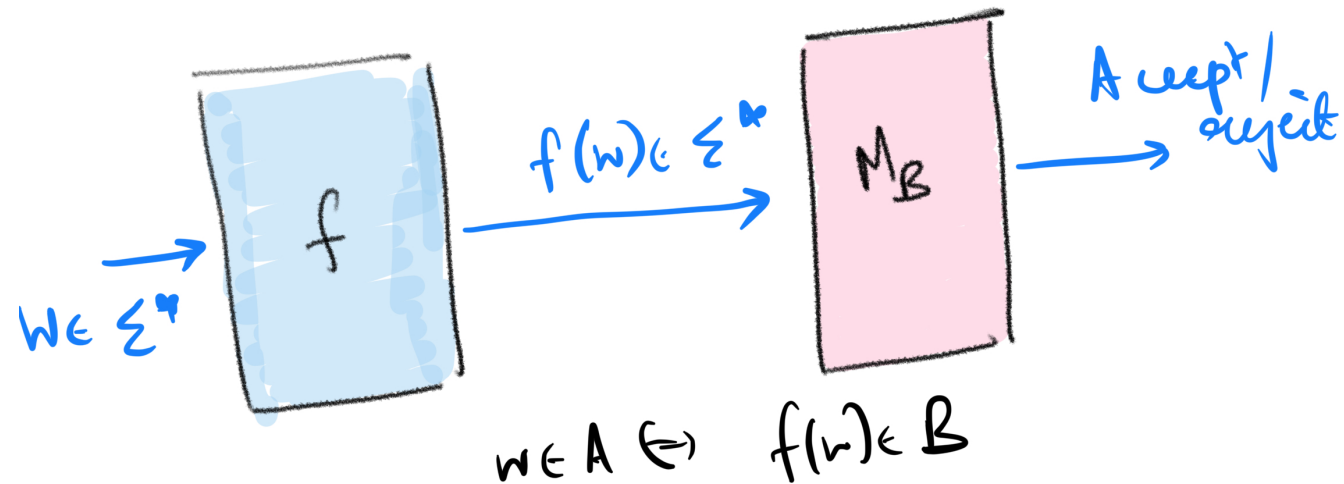
M_A accepts $w \iff M_B$ accepts $f(w)$
 $\iff f(w) \in B$
 $\iff w \in A$.



Mapping Reducibility: Properties

Theorem: If $A \leq_m B$ and B is decidable, then A is decidable.

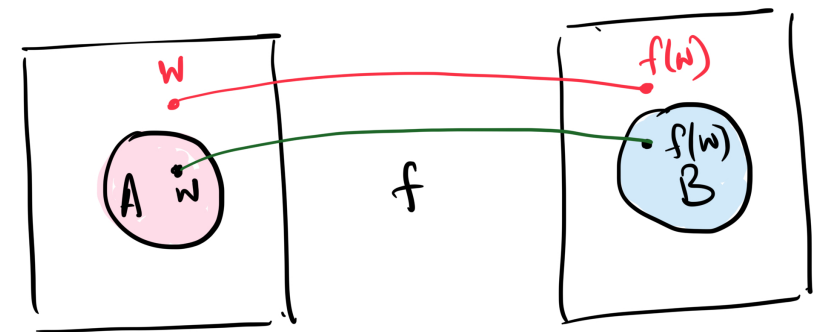
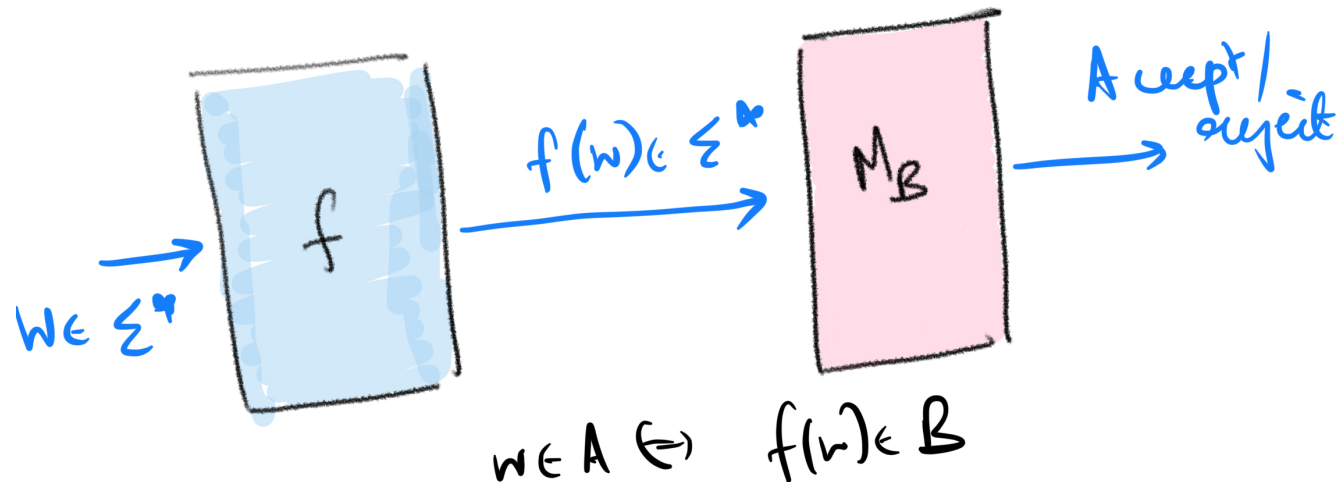
A decider M_A for A :



Mapping Reducibility: Properties

Theorem: If $A \leq_m B$ and B is decidable, then A is decidable.

A decider M_A for A :

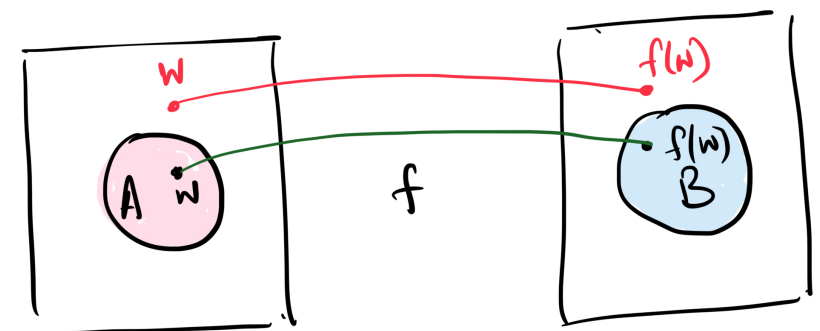
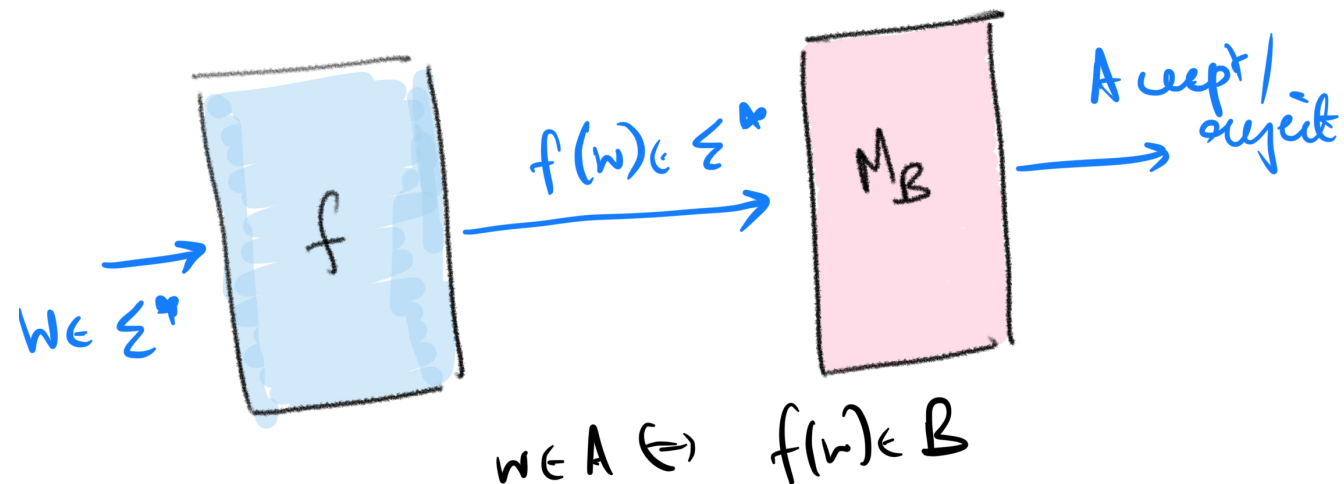


Corollary: If $A \leq_m B$ and A is undecidable, then B is undecidable.

Mapping Reducibility: Properties

Theorem: If $A \leq_m B$ and B is decidable, then A is decidable.

A decider M_A for A :



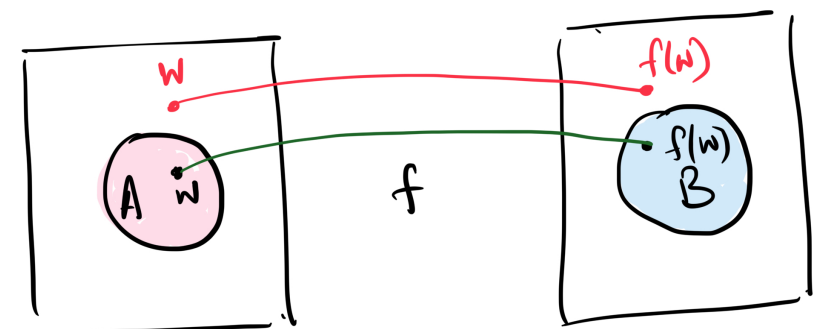
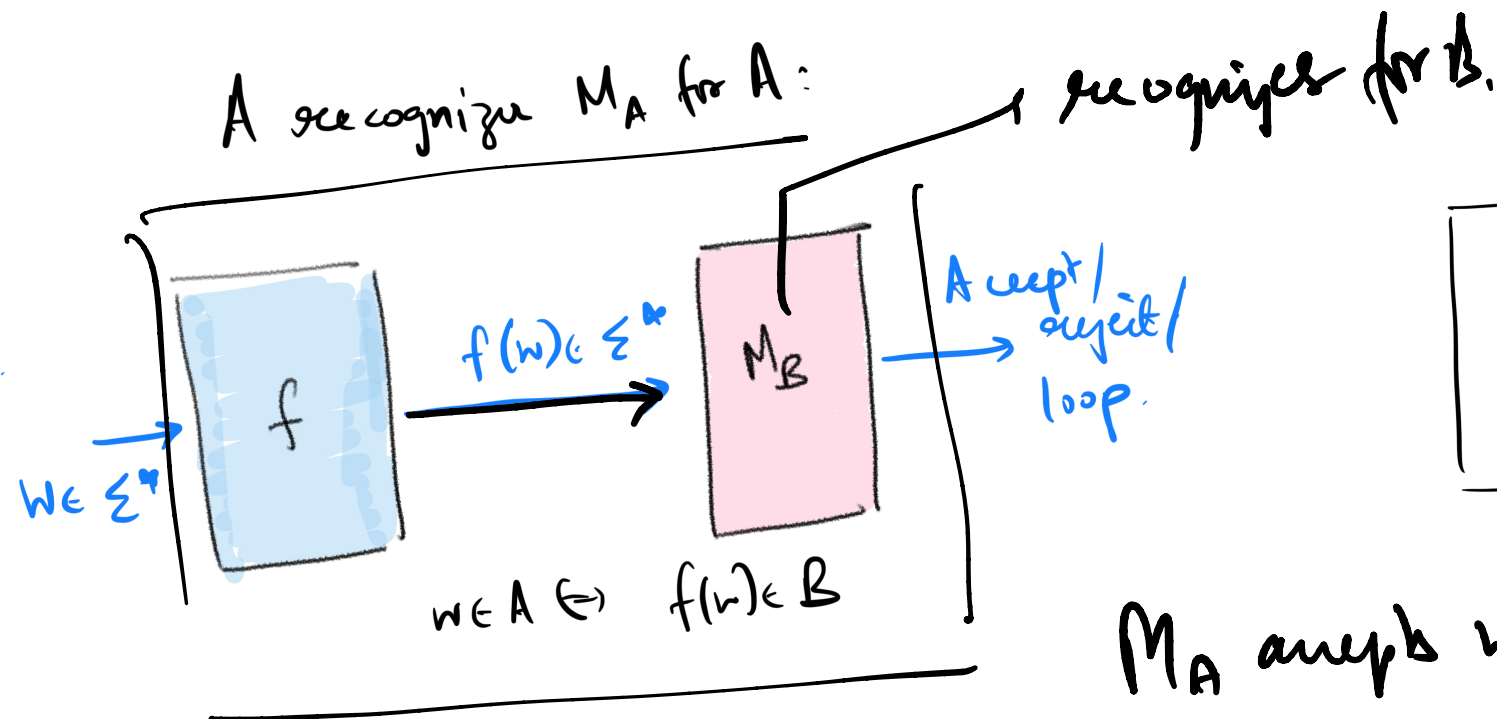
Corollary: If $A \leq_m B$ and A is undecidable, then B is undecidable.

A recipe for undecidability: To show B is undecidable, find undecidable A and a mapping reduction from A to B .

$$A_{TM} \leq_m \overline{BMPCTM}$$

Mapping Reducibility: Properties

Theorem: If $A \leq_m B$ and B is recognizable, then A is recognizable.



$M_A \text{ accepts } w \Leftrightarrow M_B \text{ accepts } f(w)$

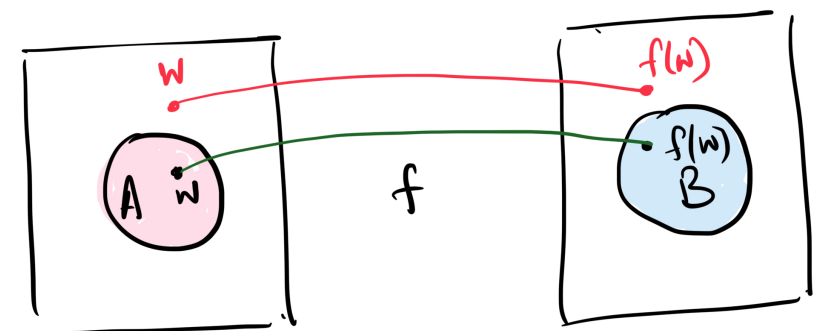
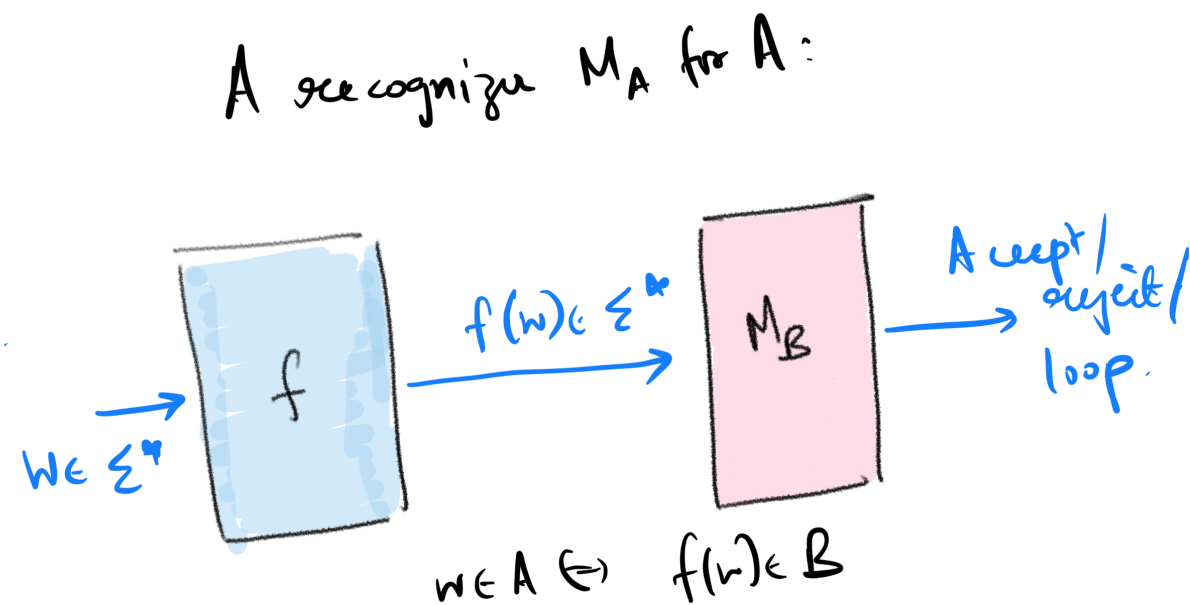
$\Rightarrow f(w) \in B$

$\Rightarrow w \in A$

mapping function

Mapping Reducibility: Properties

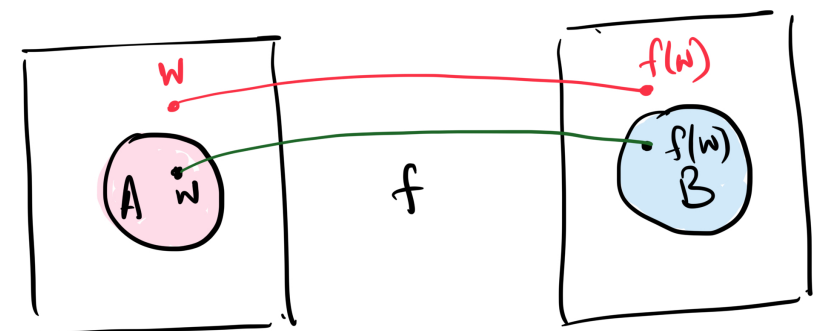
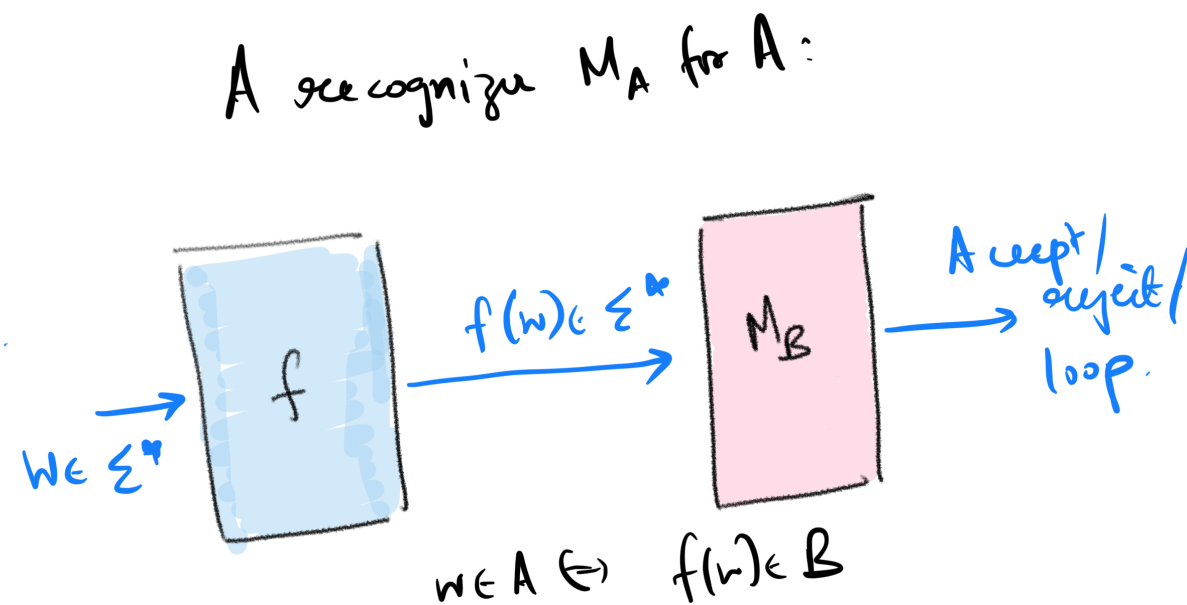
Theorem: If $A \leq_m B$ and B is recognizable, then A is recognizable.



Corollary: If $A \leq_m B$ and A is unrecognizable, then B is unrecognizable.

Mapping Reducibility: Properties

Theorem: If $A \leq_m B$ and B is recognizable, then A is recognizable.



Corollary: If $A \leq_m B$ and A is unrecognizable, then B is unrecognizable.

A recipe for unrecognizability : To show B is unrecognizable, find unrecognizable A and a mapping reduction from A to B .

$\hookrightarrow \uparrow_{TM}$

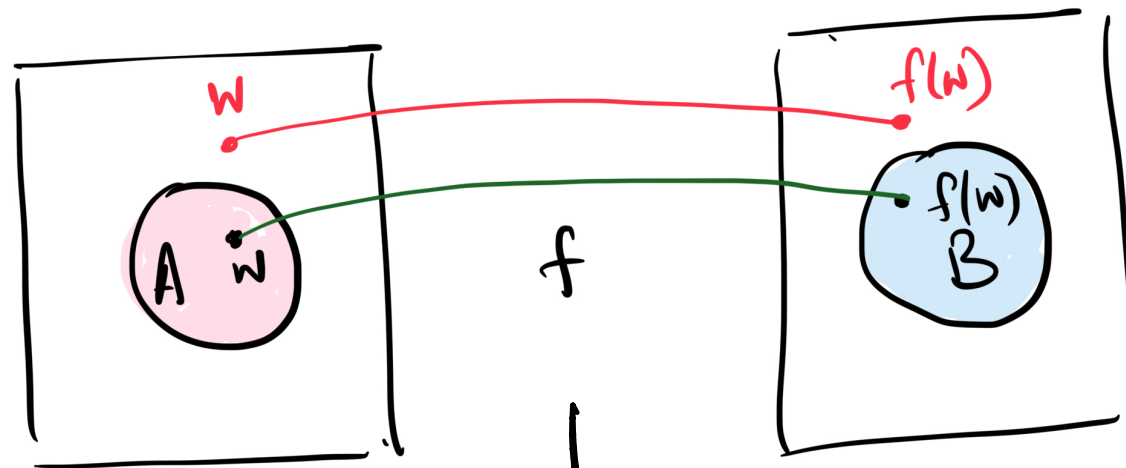
Mapping Reducibility: Properties

Mapping Reducibility: Properties

Theorem: If $A \leq_m B$ then $\bar{A} \leq_m \bar{B}$.

Mapping Reducibility: Properties

Theorem: If $A \leq_m B$ then $\bar{A} \leq_m \bar{B}$.



The same f is also a mapping reduction from $\bar{A}$ to $\bar{B}$.

as $w \notin A \Leftrightarrow f(w) \notin B$
 $\Rightarrow w \in \bar{A} \Leftrightarrow f(w) \in \bar{B}$.

Using Mapping Reduction for Unrecognizability

EMPTY_{TM} is unrecognizable

- Let $\text{EMPTY}_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$.

Theorem: EMPTY_{TM} is unrecognizable.

EMPTY_{TM} is unrecognizable

- Let $\text{EMPTY}_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$.

Theorem: EMPTY_{TM} is unrecognizable.

- **Proof:** Proof by contradiction. We will show $\bar{A}_{TM} \leq_m \text{EMPTY}_{TM}$.

EMPTY_{TM} is unrecognizable

- Let $\text{EMPTY}_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$.

Theorem: EMPTY_{TM} is unrecognizable.

- **Proof:** Proof by contradiction. We will show $\bar{A}_{TM} \leq_m \text{EMPTY}_{TM}$.

Reduction Function: $f(\langle M, w \rangle) = \langle M_w \rangle$

EMPTY_{TM} is unrecognizable

- Let $\text{EMPTY}_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$.

Theorem: EMPTY_{TM} is unrecognizable.

- Proof:** Proof by contradiction. We will show $\bar{A}_{TM} \leq_m \text{EMPTY}_{TM}$.

Reduction Function: $f(\langle M, w \rangle) = \langle M_w \rangle$

$M_w =$ “On input $x \in \{0,1\}^*$

1. If $x \neq w$, then reject.
2. Simulate M on w .

EMPTY_{TM} is unrecognizable

- Let $\text{EMPTY}_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$.

Theorem: EMPTY_{TM} is unrecognizable.

- Proof:** Proof by contradiction. We will show $\bar{A}_{TM} \leq_m \text{EMPTY}_{TM}$.

Reduction Function: $f(\langle M, w \rangle) = \langle M_w \rangle$

$M_w =$ "On input $x \in \{0,1\}^*$

1. If $x \neq w$, then reject.
2. Simulate M on w .

$$L(M_w) = \begin{cases} \{w\} & \text{if } M \text{ accepts } w \\ \emptyset & \text{if } M \text{ does not accept } w \end{cases}$$

EMPTY_{TM} is unrecognizable

- Let $\text{EMPTY}_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$.

Theorem: EMPTY_{TM} is unrecognizable.

- Proof:** Proof by contradiction. We will show $\bar{A}_{TM} \leq_m \text{EMPTY}_{TM}$.

Reduction Function: $f(\langle M, w \rangle) = \langle M_w \rangle$

$M_w =$ "On input $x \in \{0,1\}^*$

1. If $x \neq w$, then reject.
2. Simulate M on w .

$$L(M_w) = \begin{cases} \{w\} & \text{if } M \text{ accepts } w \\ \emptyset & \text{if } M \text{ does not accept } w \end{cases}$$

$$\begin{aligned} f(\langle M, w \rangle) \in \text{EMPTY}_{TM} &\iff \langle M_w \rangle \in \text{EMPTY}_{TM} \\ &\iff L(M_w) = \emptyset \\ &\iff M \text{ does not accept } w \\ &\iff \langle M, w \rangle \in \bar{A}_{TM} \end{aligned}$$

EMPTY_{TM} is unrecognizable

- Let $\text{EMPTY}_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$.

Theorem: EMPTY_{TM} is unrecognizable.

- Proof:** Proof by contradiction. We will show $\bar{A}_{TM} \leq_m \text{EMPTY}_{TM}$.

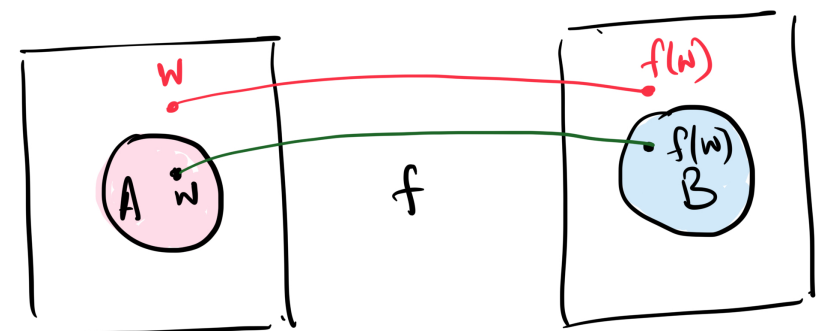
Reduction Function: $f(\langle M, w \rangle) = \langle M_w \rangle$

$M_w =$ "On input $x \in \{0,1\}^*$

1. If $x \neq w$, then reject.
2. Simulate M on w .

$$L(M_w) = \begin{cases} \{w\} & \text{if } M \text{ accepts } w \\ \emptyset & \text{if } M \text{ does not accept } w \end{cases}$$

$$\begin{aligned} f(\langle M, w \rangle) \in \text{EMPTY}_{TM} &\iff \langle M_w \rangle \in \text{EMPTY}_{TM} \\ &\iff L(M_w) = \emptyset \\ &\iff M \text{ does not accept } w \\ &\iff \langle M, w \rangle \in \bar{A}_{TM} \end{aligned}$$



Reducibility Terminology

Why do we use the term “reduce”?

- When we reduce A to B , we show how to solve A by using B . We conclude that A is no harder than B . (This suggests the $\leq_m$ notation).
- Possibility 1: We bring A 's difficulty down to B 's difficulty.
- Possibility 2: We bring B 's difficulty up to A 's difficulty.

Summary

- Introduced the mapping reducibility to show undecidability/unrecognizability.
- Showed EMPTY_{TM} is undecidable and unrecognizable.
- Next time: more examples.